TRINITY COLLEGE · OCTOBER 19, 2026

AI Integration

Vision models, and the tests

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

Where we are

Two weeks ago

Your program answered from documents
Five questions, with and without them
You approved a plan before code

This week

Your program reads a photograph
The model reads, your code checks
The agent writes the tests; you prove they ran
A hook says no on your behalf

> Stage 4 of the chain. Two more to go: review, then maintain.

The same request, with a picture in it

```
{"model": "minimax/minimax-m3",
 "messages": [{"role": "user", "content": [
    {"type": "text", "text": "Read this receipt."},
    {"type": "image_url", "image_url": {"url": "
 ]}]}
```

- `content` becomes a **list of parts**: text and images, in order.
- The image travels as a data URL (or a public `https://` URL).
- Same endpoint, same key, same two class models. Both accept images.

curl -s https://openrouter.ai/api/v1/models shows input_modalities for every model.

What it costs

	MiniMax M3	MiMo v2.6-flash
One receipt image (520×760 PNG)	~790 tokens in	~510 tokens in
Reply as JSON	230–360 out	300–400 out (one run: 1,436)
Five receipts, one eval	about $0.003	about $0.001

- An image is **tokens**: each model tiles it its own way, so the count differs.
- A 4,000-pixel phone photo costs more and reads no better. Resize.
- Each test that calls the model is a billed call: **one call per image, shared by the checks**.

Measured October 9, 2026, with the reference program; your numbers will differ a little.

What vision models read well, and where they guess

Reads well

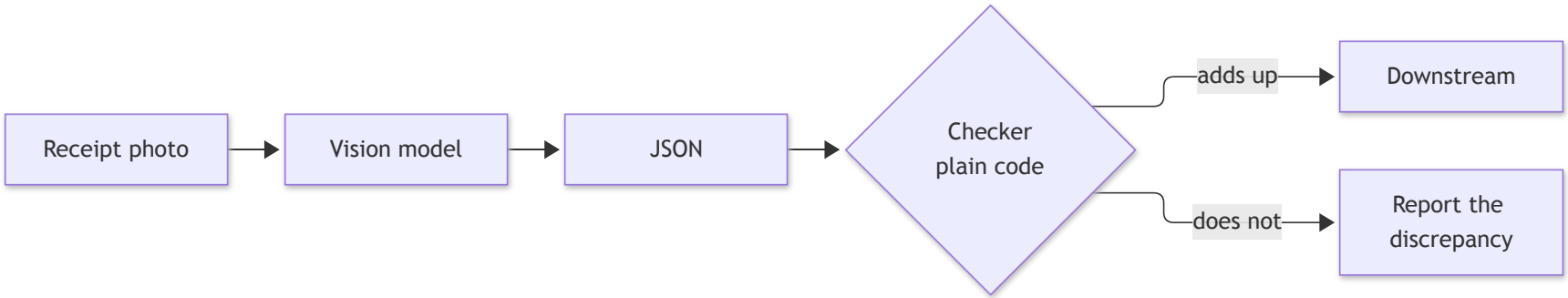
Printed text, even tilted or faded
Layout: what is a heading, a row, a total
Which of two pictures matches a description
"What is this?"

Guesses

Exact digits in small or blurred print
Counts above a handful
A year that is not printed
Anything that *should* be there

> A vision model returns plausible numbers. Plausible is not the same as right.

The model reads. The code checks.



- The checker knows arithmetic: items × quantity = subtotal; subtotal − discount + tax = total.
- It knows the schema: a date is `YYYY-MM-DD` or null; a total is a number.
- It **reports**; it does not repair. A fix belongs in the prompt, the spec, or a person's hands.

Week 3's rule again: a reply you cannot parse is a failed case, not a crash.

Live: one receipt

```
export CHAT_MODEL=minimax/minimax-m3
python3 extract.py images/receipt-01.png
```

```
{"merchant": "Mather Dining Hall", "date": "2026-10-02", "currency": "USD",
 "items": [{"name": "Chicken wrap", "qty": 1, "price": 7.49}, ...],
 "subtotal": 12.84, "discount": null, "tax": 0.82, "total": 13.66}
CHECK: ok
[minimax/minimax-m3 · 793 in / 235 out]
```

Live: the one that fails

```
python3 extract.py images/receipt-03.png     # coffee, with a student discou
```

```
 "subtotal": 11.5, "discount": -1.15, "tax": 0.66, "total": 11.01
CHECK: subtotal - discount + tax = 13.31, total says 11.01
```

- The model copied the minus sign **as printed**. The checker expected a positive discount.
- The read is right; the spec was ambiguous. MiMo returned `1.15` and passed.

> Was the model wrong, or the spec? Decide, write it down, then fix the spec first.

Chain stage 4: the tests

Stage	Artifact	Written by	Approved by
Plan	`intent/*.md`	agent, after interviewing you	you
Design	`spec.md`	agent, from the intent	you, against the intent
Build	`plan.md`, then code	agent	you, before any code
Test	**tests, lint, the eval**	**agent**	**you confirm the loop ran**

- The agent writes the tests. That is not the problem.
- The problem is believing "all tests pass" without looking.

Two kinds of tests

Need no key

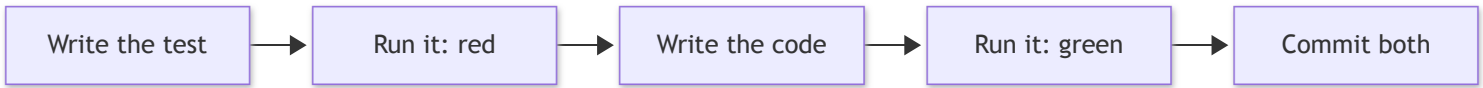
The checker on hand-written receipts
The JSON parser on fenced and prose replies
The image encoder’s MIME type
Run in 0.1 s, every time, for free

Need a key

The eval: every image in `images/` against `expected.json`
total, item count, merchant, date, arithmetic
One model call per image, billed

> If every test needs the key, the agent will stop running them.
>
> Keep the fast ones fast.

Tests first, then code, then the output



- The first run **must fail**. A test that passes before the code exists tests nothing.
- The prompt: *"write the tests first and run them so I see them fail, then write the code and run them again."*
- The commit shows the test and the code together. `CHECKS.md` holds the output.

Verifying that the loop ran

- "All tests pass" is a claim. The **output** is evidence. Read the summary line: how many, how long, any skipped.
- `-v` names each test. A suite that is all `skipped` passed nothing.
- The commit history shows the test **before** the code it tests.
- A linter ran: `ruff check .`, or the build's lint task. Record what it found, even "nothing."

> You are not re-running the tests. You are checking that they were run, on this code, and what they said.

The test that passes for the wrong reason

- The agent edits `expected.json` to match the output.
- The agent adds `if name == "receipt-01": return 13.96`.
- The agent marks the test `skip` "temporarily."
- The agent deletes the test.

> REVIEW.md, pass 1: tests deleted, skipped, or weakened to make the build pass must not merge.

A hook is a rule the agent cannot forget

```
{"hooks": {"PreToolUse": [
  {"matcher": "Edit|Write|MultiEdit|Bash",
   "hooks": [{"type": "command",
              "command": "python3 .claude/hooks/guard.py"}]}]}}
```

- Runs **before** every matching tool call, with the call as JSON on stdin.
- Exit **2** blocks the call; stderr goes back to the agent as the reason.
- Exit 0 allows. Exit 1 means "the hook broke" and the call goes through: a guard fails **open**.
- `.claude/settings.json` is committed; your key stays in `settings.local.json`.

Chapter 6 of Claude Code: Up and Running · code.claude.com/docs/en/hooks

The guard: three rules

Blocked	Because
Edit or Write to `images/` or `expected.json`	the answer key: a test that passes because the key changed proves nothing
`rm -r`, `git push --force`, `git reset --hard`, `git checkout --`, `git clean`	destructive; a person runs those
`pip install` / `uv add` / `npm install` of a package not in `spec.md`	the course rule, enforced: name it with a reason first

Test it without the agent: pipe a tool call as JSON into guard.py and read the exit code.
examples/guardrail-hook/README.md

Live: ask the agent to cheat

```
In images/expected.json change receipt-01's total from 13.66 to 13.96.
```

```
A hook blocked the edit. Its exact message:
  images/expected.json is the answer key. Fix the code, not the
  expected values. If you believe the key is wrong, say so and stop.
I'm stopping. expected.json is unchanged.
```

> The message names the fix. Write hook messages for the agent that will read them.

A browser-driven check

```
page.goto("http://localhost:8000")
assert "sk-or" not in page.content()                    # no key in the page
page.fill("#q", "Reply with the single word PONG.")
page.click("form button")
page.wait_for_function(
    "document.querySelector('#out').textContent.length > 1")
assert all(auth is None for _, auth in requests)        # no auth header
assert all("openrouter" not in u for u, _ in requests) # never called
```

- Playwright drives a real Chromium. It sees what the browser sent, not what the code meant.
- A unit test of the server can’t see this. A person clicking can’t prove it.

examples/key-on-server/test_browser.py · pip install playwright pytest · playwright install chromium

Team Project 1 by October 26

Test stage	What is due
Agent-written tests	the checker-style tests for your pipeline, plus an eval on real inputs
Browser-driven check	the front end: generate, display, critique shows up, no key in the browser
The loop ran	`CHECKS.md` with the output, and the commits that show tests before code

- The vision critique in TP1 is this week's lab in your app: the model reads the generated image, your code checks the critique has the fields the spec names.

Lab: receipts to data

0 Repo, images, **the guard**, `CLAUDE.md`

1 Intent and spec, briefly

2 The plan, proof split in two

3 **Tests first**, then code; two models

4 Break the loop on purpose

5 Your own receipts, at home

weeks/05/lab.md · 40 minutes · Prof. Kousen circulates

Save evidence of the work

```
git add .
git commit -m "Receipt reader: tests, two models, own receipts"
git tag week05-submitted
git push -u origin main
git push origin week05-submitted
```

Tests before code in the history. `CHECKS.md`: both models, the linter, the step-4 story. `images/expected.json` unchanged.

Before October 26

- **Week 5 submission:** the lab, with two of your own receipts.
- **Team Project 1, Test stage:** tests that run, and the browser check.
- Week 6 is the **mid-term**: image generation, and the Deploy stage, with a separate reviewing agent and `REVIEW.md`.
- The SDLC Playbook certificate is assigned next week.

Before you leave

- What does the checker know that the model doesn’t?
- What would you look at to confirm the tests ran?
- Why does a hook that crashes let the call through?

Sources and next steps

- OpenRouter: images in requests · Playwright for Python · Claude Code hooks.
- Ken Kousen, *Claude Code: Up and Running*, Chapter 6 (hooks).
- The AI-Native SDLC Playbook: the Test stage.
- Artifact-chain template: `REVIEW.md` pass 1.
- Reference program, tests, guard, and browser check rehearsed October 9, 2026 on `minimax/minimax-m3` and `xiaomi/mimo-v2.6-flash`.

Prepared October 9, 2026.